

Instructions for Submission

1. File Naming and Submission:
 - a. Create a **ZIP file** containing all your solution files.
 - b. Name the ZIP file as lab06_<ERP>.zip, where <ERP> is your ERP.
 - c. Ensure that the ZIP file contains:
 - i. C++ source files
 - ii. No unnecessary files (such as executables, temporary files, or editor configs)
 - d. Do not include TEXT(.txt) files.
2. Code Neatness:
 - a. Ensure your code is **well-formatted** and easy to read. Use proper **indentation** and **meaningful variable names**.
 - b. Include appropriate **comments** to explain the logic where necessary (especially for functions and complex sections of the code).
 - c. **Add clear explanations where required (or specified)** to make the logic of your program easy to follow.
3. Code Compilation and Functionality:
 - a. Ensure your code compiles without errors or warnings. You should be able to compile and run your code on any system with a standard C++ compiler (e.g., GCC, Clang, MSVC).
 - b. Double-check that the functionality of the program meets the requirements outlined in the task.
4. Additional Instructions:
 - a. Do not use any libraries or features that have not been covered in the course material, unless specifically instructed.
 - b. Make sure your code adheres to the principles of good software development (e.g., minimal repetition, and no hard-coding).
 - c. If you encounter any issues or require clarifications, reach out before the submission deadline.
5. Deadline:
 - a. Ensure that you submit the assignment on time. Late submissions will not be accepted.
 - b. Double-check that the zip file contains all required files before submitting.
6. Plagiarism Policy:
 - a. The work submitted must be entirely your own. Any code or content copied from others (including online sources, classmates or AI) will result in an automatic zero for the task.

Lab Tasks

Task 1: Row-wise Number Distribution

Write a program that reads a sequence of integers and a number of rows from the user. The program should then print the numbers distributed across the specified number of rows in a round-robin pattern.

Each number from the input sequence is placed in a row based on its position. The first number goes to row 1, the second to row 2, and so on, wrapping around after the last row. All numbers in the same row should be printed on the same line.

Input:

- An integer N representing how many numbers to read.
- N integers entered by the user.
- An integer R representing the number of rows.

Output:

- The numbers distributed across R rows, with each row's numbers printed on a separate line, separated by spaces.

Example:

```
Enter number of integers: 12
Enter 12 integers: 10 20 30 40 50 60 70 80 90 100 110 120
Enter number of rows: 3
10 40 70 100
20 50 80 110
30 60 90 120

Enter number of integers: 8
Enter 8 integers: 1 2 3 4 5 6 7 8
Enter number of rows: 4
1 5
2 6
3 7
4 8
```

Hint: Use nested loops and modular arithmetic. The outer loop iterates through each row (0 to R-1), and the inner loop goes through all numbers, printing only those whose position modulo R matches the current row. Avoid using arrays or vectors for storage.

Task 2: Left and Right Rotation of an Array

Write a program that performs both left and right rotations on a fixed-size array of 10 integers. The program should take three inputs:

- 10 integer values to fill the array.
- A character ('L' for left or 'R' for right) representing the rotation direction.
- An integer N representing how many positions to rotate the array.

Perform the rotation in place, without using any additional arrays or data structures.

- Each **left rotation** moves the first element to the end, shifting all other elements one position to the left.
- Each **right rotation** moves the last element to the beginning, shifting all other elements one position to the right.

Example

```
Enter 10 integers: 1 2 3 4 5 6 7 8 9 10
Enter rotation direction (L/R): L
Enter number of rotations: 3
Array after 3 left rotations:
4 5 6 7 8 9 10 1 2 3

Enter 10 integers: 1 2 3 4 5 6 7 8 9 10
Enter rotation direction (L/R): R
Enter number of rotations: 2
Array after 2 right rotations:
9 10 1 2 3 4 5 6 7 8
```

Hint: Use a single temporary variable to hold either the first element (for left rotation) or the last element (for right rotation) during each rotation. Use a loop to shift the remaining elements accordingly. Implement the logic using if-else or switch statements to handle both directions.

Task 3: Matrix Addition Using 1D Arrays

Matrix addition is a fundamental operation where corresponding elements of two matrices are added together. In this task, you will implement it using one-dimensional arrays (or **std::vector**) instead of two-dimensional ones. This will help you understand how multidimensional data can be represented and accessed in linear memory.

Your program should work with two hardcoded 3×3 matrices:

- Matrix A of size 3×3
- Matrix B of size 3×3

Store both matrices as one-dimensional arrays (**std::vector<int>**). Then add them to produce a resulting matrix C of size 3×3 , also represented as a 1D array.

Each element of the result is computed by adding corresponding elements:

$$C[i][j] = A[i][j] + B[i][j]$$

Since the matrices are stored in one-dimensional arrays, use index mapping to access elements:

$$A[i][j] \rightarrow A[i \times 3 + j]$$

$$B[i][j] \rightarrow B[i \times 3 + j]$$

$$C[i][j] \rightarrow C[i \times 3 + j]$$

After computing the result, print all three matrices (A, B, and C) in a formatted 3×3 layout, where each row is displayed on a separate line.

Example

Matrix A (3x3):

```
1 2 3
4 5 6
7 8 9
```

Matrix B (3x3):

```
9 8 7
6 5 4
3 2 1
```

Resulting matrix C (3x3):

```
10 10 10
10 10 10
10 10 10
```